



Navneet Verma

Posted on 10 de jul.



6



3



1



1



2

The Complete TypeScript Mastery Guide

[#typescript](#) [#webdev](#) [#systemdesign](#) [#tutorial](#)

Learn TypeScript From First Principles to Senior/Staff-Level Production Engineering

If you searched for how to learn TypeScript properly — not just the syntax, but the thinking behind it — this guide is built for that. Most TypeScript tutorials stop at "here's an interface, here's a generic." This one goes further: it's a single, exhaustive TypeScript tutorial and reference that walks through the type system, object-oriented programming, generics, async programming, design patterns, SOLID and DRY principles, error handling, testing, and the tooling that real production teams run in CI — the same TypeScript best practices used at top-tier engineering organizations.

Whether you're a beginner looking for a structured TypeScript for beginners path, or an experienced JavaScript developer making the jump to advanced TypeScript and system design, you can read this end to end or jump straight to the section you need using the linked table of contents below.

Table of Contents

- 1. Introduction — What Is TypeScript & Why It Exists
 - 2. Installation, Setup & `tsconfig.json` Deep Dive
 - 3. Variables & the Complete Type System
 - 4. Functions — Every Form, Overloads, `this`, and Best Practices
 - 5. Arrays & Tuples
 - 6. Objects & Type Aliases
 - 7. Interfaces
 - 8. Enums & Literal Types
 - 9. Union, Intersection & Discriminated Unions
 - 10. Type Narrowing, Assertions & Type Guards
 - 11. Classes & Object-Oriented Programming
 - 12. Generics — Basic to Advanced
 - 13. Modules, Namespaces & Project Structure
 - 14. Asynchronous Programming — Event Loop to Production Patterns
 - 15. Advanced/Utility Types & the Type-Level Programming Toolkit
 - 16. Design Patterns in TypeScript
 - 17. SOLID, DRY, KISS, YAGNI — Principles Applied With Real TS Code
 - 18. Error Handling Strategies
 - 19. Testing TypeScript
 - 20. Tooling, Linting, Build Systems & CI/CD for Production TS
 - 21. Performance, Compiler Internals & Scaling Large Codebases
 - 22. Interview Cheat Sheet (Expanded)
 - 23. One-Page Quick Revision Sheet
-

1. Introduction — What Is TypeScript & Why It Exists

1.1 Definition

TypeScript (TS) is an open-source language developed and maintained by Microsoft. It is a **strict syntactical superset of JavaScript** — every valid JavaScript program is also a valid TypeScript program — that adds an optional **static structural type system** on top of JS semantics. TypeScript never changes JavaScript's runtime behavior; it only adds a compile-time layer that is erased before the code runs.

Key mental model: **TypeScript types are a development-time tool**. At runtime, there is no TypeScript — only plain JavaScript. This has a critical implication:

TypeScript can be *lied to* (e.g., via `as`, `any`, unsafe casts) and it cannot protect you against bad data arriving from the network, disk, or user input unless you validate it (see §18 on runtime validation).

1.2 Why TypeScript? (Beyond the Basics)

Reason	Explanation
Compile-time error detection	Type mismatches, typos in property names, wrong argument counts, and null/undefined misuse are caught before shipping.
Self-documenting code	Function signatures and interfaces act as always-up-to-date documentation, unlike comments which rot.
Refactoring safety	Renaming a property or changing a function signature immediately surfaces every call site that breaks, across a codebase of any size.
IDE tooling / IntelliSense	Autocomplete, jump-to-definition, and inline docs work reliably because the editor understands your data shapes.
Scales with team size	In large teams, types act as a contract between modules/services owned by different people, reducing "what shape does this object have?" questions.
Gradual adoption	You can introduce TS file-by-file into an existing JS codebase (<code>allowJs</code> , <code>checkJs</code>), making it low-risk to adopt incrementally.
Ecosystem	Virtually every major library ships type definitions (<code>@types/*</code> or bundled <code>.d.ts</code> files), giving you typed access to the entire npm ecosystem.

1.3 JavaScript vs TypeScript — Concrete Comparison

```
// JavaScript – no static checks, error only discovered at runtime (or never, silent)
let user = { name: "Asha", age: 25 };
user.age = "twenty five"; // no error, silently corrupts data

// TypeScript – the type annotation creates a compile-time contract
let typedUser: { name: string; age: number } = { name: "Asha", age: 25 };
typedUser.age = "twenty five"; // Incorrect – compile error:
// Type 'string' is not assignable to type 'number'.
```

1.4 How TypeScript Works Internally (Compiler Pipeline)

Understanding the pipeline is what separates "I use TS" from "I understand TS":

1. **Parsing** — Source `.ts` text is tokenized and parsed into an Abstract Syntax Tree (AST).
2. **Binding** — The binder walks the AST and creates symbols (a "Symbol" is TypeScript's internal representation of a named entity — variable, function, class, etc.) and builds scopes.
3. **Type Checking** — The type checker walks the AST, resolves types (via inference and annotations), and validates every expression against the type rules. This is where 95% of "the compiler complains" moments come from.
4. **Emit** — Type annotations are *stripped* and the AST is transformed/down-levleled to the JavaScript target (e.g., ES2020, CommonJS/ESM modules) and written to `.js` files. Optionally, `.d.ts` declaration files and source maps are emitted.

Because type erasure happens at step 4, **types cannot be used for runtime logic** (e.g., you cannot do `if (x instanceof SomeInterface)` — interfaces don't exist at runtime; only classes, which compile to real JS constructor functions, support `instanceof`).

1.5 The Learning & Production Roadmap

Practice discipline that senior engineers follow:

- Enable `strict: true` from day one of a project — retrofitting strict mode onto a large loose codebase is far more expensive than starting strict.
- Treat the type system as executable documentation: if a reviewer has to ask "what shape is this?", the types are incomplete.
- Never treat `any` as a shortcut to "make the red squiggly go away" — it is a designed escape hatch to be used sparingly and explicitly, not a default.

2. Installation, Setup & `tsconfig.json` Deep Dive

2.1 Prerequisites

1. **Install Node.js** (LTS version) from <https://nodejs.org> — TypeScript's compiler (`tsc`) runs on Node.
2. **Install TypeScript:**

```
npm install -g typescript    # global CLI (fine for learning)
npm install -D typescript    # project-local (required for production – pins ve
tsc -v                      # verify installation
```

Production best practice: always install TypeScript as a local `devDependency`, never rely on a global install. This guarantees every developer and every CI runner compiles with the exact same compiler version, avoiding "works on my machine" type-checking drift.

1. Initialize configuration:

```
npx tsc --init
```

This generates `tsconfig.json`, the single source of truth for how the compiler behaves.

2.2 Compiling & Running

```
tsc                # compiles the whole project per tsconfig.json
tsc app.ts         # compiles a single file, ignoring tsconfig module resolution nu
tsc -w            # watch mode – recompiles on save; use during local development
node dist/app.js   # run the emitted JavaScript
```

For iterative development, most production teams skip the manual `tsc` + `node` cycle and use `ts-node`, `tsx`, or a bundler (`esbuild`, `swc`, `vite`) for near-instant feedback, then use `tsc --noEmit` purely as a type-checking gate in CI (bundlers strip types without checking them — `tsc` is what catches errors).

2.3 `tsconfig.json` — Every Option That Matters in Production

```
{
  "compilerOptions": {
    /* --- Type Checking (strictness) --- */
    "strict": true,                // umbrella flag – turns on all strict fla
    "noImplicitAny": true,         // (included in strict) disallow untyped `
    "strictNullChecks": true,     // (included in strict) null/undefined are
    "strictFunctionTypes": true,   // (included in strict) sound function par
    "strictBindCallApply": true,   // (included in strict) type-check .bind/.
    "strictPropertyInitialization": true, // (included in strict) class fields must
    "noImplicitThis": true,        // (included in strict) error on `this` wi
    "alwaysStrict": true,          // (included in strict) emit "use strict"
    "noUncheckedIndexedAccess": true, // index signature access returns `T | und
    "exactOptionalPropertyTypes": true, // `foo?: string` means "absent or string"
```

```

"noImplicitOverride": true,           // require explicit `override` keyword whe
"noFallthroughCasesInSwitch": true,  // catch missing `break` in switch stateme
"noUnusedLocals": true,              // flag unused local variables
"noUnusedParameters": true,         // flag unused function parameters

/* --- Modules & Interop --- */
"module": "NodeNext",               // or "ESNext" for bundler-based frontends
"moduleResolution": "NodeNext",     // matches Node's actual ESM/CJS resolutio
"esModuleInterop": true,            // allows `import express from "express"`
"resolveJsonModule": true,          // allows `import data from "./data.json"`
"isolatedModules": true,            // ensures every file can be transpiled in
"verbatimModuleSyntax": true,       // forces explicit `import type` for type-

/* --- Emit --- */
"target": "ES2022",                 // JS language level of the OUTPUT
"outDir": "./dist",                 // compiled output folder
"rootDir": "./src",                 // source folder (enforces output mirrors
"declaration": true,                // emit .d.ts files – REQUIRED if you publ
"declarationMap": true,              // enables "go to definition" into your TS
"sourceMap": true,                  // enables debugger breakpoints in origina
"removeComments": false,            // keep comments (JSDoc) in emitted .d.ts

/* --- Interop / Safety --- */
"skipLibCheck": true,               // skip type-checking .d.ts files of depen
"forceConsistentCasingInFileNames": true, // avoid cross-OS casing bugs (macOS/w
"allowJs": false,                   // set true only during incremental JS→TS
"checkJs": false,                   // set true alongside allowJs to type-chec

/* --- Paths --- */
"baseUrl": ".",
"paths": { "@/*": ["src/*"] }      // path aliases – must be mirrored in your
},
"include": ["src/**/*"],
"exclude": ["node_modules", "dist", "**/*.spec.ts"]
}

```

Senior-level tsconfig guidance:

- Never ship a production repo without `strict: true`. Anything less is an unfinished migration, not a design choice.
- `noUncheckedIndexedAccess` is one of the highest-value flags most teams forget — without it, `arr[i]` and `obj[key]` are typed as if they always succeed, which is false and a common source of runtime `undefined` bugs.
- For monorepos, use **TypeScript Project References** (`"composite": true`, `"references": [{ "path": "../shared" } ]`) so `tsc --build` only recompiles changed

packages — this is what makes large monorepos compile fast.

- Keep `tsconfig.base.json` at the repo root with shared strict settings, and let each package `extend` it — this is the DRY principle applied to configuration.

3. Variables & the Complete Type System

3.1 `let`, `const`, and (avoid) `var`

```
let score: number = 98.6; // reassignable, block-scoped
const PI: number = 3.14159; // NOT reassignable, block-scoped
var legacy = "avoid"; // function-scoped, hoisted, error-prone – do not use
```

Production rule: default to `const`. Only use `let` when a binding is genuinely reassigned (loop counters, accumulator variables, values reassigned in conditional branches). `var` should never appear in a modern TypeScript codebase — enforce this with the ESLint rule `no-var`.

3.2 Primitive Types

Type	Meaning	Example
<code>string</code>	Text	<code>let city: string = "Hyderabad";</code>
<code>number</code>	All numbers (int & float — TS/JS do not distinguish)	<code>let price: number = 98.6;</code>
<code>boolean</code>	<code>true</code> / <code>false</code>	<code>let active: boolean = true;</code>
<code>bigint</code>	Arbitrary-precision integers	<code>let id: bigint = 12345678901234567890n;</code>
<code>symbol</code>	Unique, immutable identifiers	<code>let key: symbol = Symbol("id");</code>
<code>null</code>	Intentional absence of a value	<code>let data: null = null;</code>
<code>undefined</code>	Declared but not yet assigned	<code>let value: undefined;</code>

3.3 The Special Types — `any`, `unknown`, `void`, `never`

```
// any – completely disables type checking for that value. AVOID.
let danger: any = "hello";
danger.toUpperCase().thisMethodDoesNotExist(); // compiles fine, crashes at runtime
```



```
// unknown – the type-safe counterpart to any. Anything can be assigned TO it,
// but it cannot be USED until narrowed. This is the correct type for
// "data I don't yet know the shape of" (e.g., API responses, JSON.parse results).
let result: unknown = fetchSomething();
if (typeof result === "string") {
  console.log(result.toUpperCase()); // Correct – safe – narrowed
}

// void – a function that returns nothing meaningful.
function logMessage(msg: string): void {
  console.log(msg);
}

// never – a value that can never occur: functions that always throw,
// or infinite loops, or the empty branch of an exhaustive union.
function fail(message: string): never {
  throw new Error(message);
}
```

any vs **unknown** — the single most important type-safety decision in TS. **any** opts a value entirely out of the type system (bidirectionally assignable to and from anything). **unknown** opts *in* to safety — it can receive anything, but you must prove its shape (via `typeof`, `instanceof`, a type guard, or a schema validator) before you can operate on it. **Rule for production code: ban `any` via ESLint (`@typescript-eslint/no-explicit-any`), and use `unknown` at every true boundary (API responses, `JSON.parse`, third-party untyped libraries, `catch` clause errors).**

3.4 Type Inference vs Explicit Annotation

```
let inferred = "hello"; // TS infers `string` – no annotation needed
let explicit: string = "hi"; // explicit annotation – useful for function signature
```

Guideline: let TypeScript infer types for local variables (`let x = 5`), but *always* explicitly annotate function parameters and exported/public function return types. Inference on locals reduces noise; explicit annotations on public boundaries prevent "inference drift," where a well-intentioned internal refactor silently changes a public contract.

3.5 `null` vs `undefined` in Strict Mode

With `strictNullChecks` (part of `strict: true`), `null` and `undefined` are **not** automatically part of every type — they must be explicitly included:


```
let name: string;
name = null; // Incorrect - `null` is not assignable to `string`

let maybeName: string | null = null; // Correct - explicit opt-in
```

This single flag eliminates the majority of "cannot read property of undefined" runtime crashes that plague plain JavaScript, by forcing every possibly-absent value to be handled at the type level.

4. Functions — Every Form, Overloads, `this`, and Best Practices

4.1 Function Declaration Syntax

```
function functionName(param1: Type1, param2: Type2): ReturnType {
  // body
  return value;
}

function add(a: number, b: number): number {
  return a + b;
}
```

4.2 Return Types

Always annotate the return type of exported functions. It (a) documents intent, (b) prevents the function body from accidentally widening the return type during a refactor, and (c) speeds up compilation because TS doesn't need to infer it.

```
function greet(name: string): string {
  return `Hello, ${name}`;
}
```

4.3 Optional & Default Parameters

```
function greetUser(name: string, age?: number): string {
  return age ? `Hello ${name}, age ${age}` : `Hello ${name}`;
}

function multiply(x: number, y: number = 1): number {
  return x * y;
}
```

```
multiply(5);    // 5 (y defaults to 1)
multiply(5, 3); // 15
```

Rule: optional parameters (?) and parameters with defaults must always come *after* required parameters — TypeScript enforces this because JS call sites are positional.

4.4 Rest Parameters

```
function sumAll(...nums: number[]): number {
  return nums.reduce((total, n) => total + n, 0);
}
sumAll(1, 2, 3);    // 6
sumAll(5, 10, 15, 20); // 50
```

4.5 Arrow Functions

```
const square = (x: number): number => x * x;
const add = (a: number, b: number): number => a + b;
const isAdult = (age: number): boolean => {
  return age >= 18;
};
```

Arrow functions do not bind their own `this` — they capture `this` lexically from the enclosing scope. This makes them the correct default choice for callbacks (event handlers, array methods, promise `.then` chains) where you want `this` to refer to the surrounding class instance rather than the caller.

4.6 Typing `this` Explicitly

```
interface Button {
  label: string;
  onClick(this: Button, event: MouseEvent): void;
}
```

The `this` parameter is a compile-time-only fictitious first parameter — it's erased at emit and exists purely so the compiler can validate how the function is intended to be invoked.

4.7 Function Overloads

Overloads let a single function have multiple valid call signatures with return types that depend on the input — something a plain union parameter type cannot express precisely.

```
function parseInput(input: string): string[];
function parseInput(input: number): number[];
function parseInput(input: string | number): string[] | number[] {
  if (typeof input === "string") return input.split("");
  return [input];
}

const a = parseInput("abc"); // typed as string[]
const b = parseInput(123);   // typed as number[]
```

4.8 Function Types as Values

```
type MathOperation = (a: number, b: number) => number;

const subtract: MathOperation = (a, b) => a - b;

function applyOperation(a: number, b: number, op: MathOperation): number {
  return op(a, b);
}
```

Defining a `type` alias for a callback signature (rather than inlining it everywhere it's used) is a direct application of **DRY** — the shape is defined once and reused across every consumer.

4.9 Production Best Practices for Functions

- Keep functions small and single-purpose (this is the **Single Responsibility Principle** applied at the function level — see §17).
- Prefer pure functions (output depends only on input, no side effects) wherever business logic doesn't strictly require I/O — they are trivially testable and safely composable.
- Avoid boolean parameter flags that change behavior (`function save(data, force: boolean)`) — they force the reader to look up the function definition to understand a call site. Prefer an options object: `function save(data, options: { force: boolean })`.
- Use rest parameters instead of `arguments` (which is untyped and excluded from arrow functions entirely).
- Never mutate parameters; treat all function inputs as read-only unless the function's entire purpose is to mutate (and even then, prefer explicit "Builder"/"Mutator" naming to signal it).

5. Arrays & Tuples

5.1 Arrays

```
let numbers: number[] = [1, 2, 3, 4, 5];
let names: Array<string> = ["Asha", "Ravi", "Kiran"]; // equivalent generic syntax
let flags: boolean[] = [true, false, true];

console.log(numbers[0]); // 1
names.push("Meera");
numbers.pop();
```

`T[]` and `Array<T>` are 100% equivalent — the generic form is preferred when `T` itself is a complex union, e.g., `Array<string | number>` reads more clearly than `(string | number)[]` in some style guides, but both are correct.

5.2 Readonly Arrays

```
const readonlyNumbers: ReadonlyArray<number> = [10, 20, 30];
// equivalently: const readonlyNumbers: readonly number[] = [10, 20, 30];

readonlyNumbers[0]; // Correct — 10
readonlyNumbers.push(40); // Incorrect — push does not exist on ReadonlyArray
```

Production guidance: default function parameters that receive arrays to `readonly T[]` unless the function's job is specifically to mutate the caller's array. This prevents an entire category of "function silently mutated my array" bugs and clearly communicates intent at the type level — this is a direct application of the principle of least privilege / immutability-by-default.

5.3 Tuples

Tuples are fixed-length, fixed-order arrays where each position has its own type — ideal for representing small, structurally-fixed records like coordinate pairs or `[key, value]` entries.

```
let user: [number, string, boolean] = [1, "Mahesh", true];
console.log(user[0]); // 1 (number)
console.log(user[1]); // "Mahesh" (string)
```

5.4 Named Tuple Members (modern TS)

```
type HttpResponseTuple = [status: number, body: string, headers: Record<string, stri
```

Naming tuple elements doesn't change runtime behavior but massively improves editor tooltips and readability at call sites.

5.5 Optional & Variadic (Rest) Tuple Elements

```
type Point2D3D = [x: number, y: number, z?: number];           // optional element
type StringsThenNumbers = [string, ...number[]];              // rest element
type ConcatResult<A extends unknown[], B extends unknown[]> = [...A, ...B]; // spread
```

5.6 Nested Arrays (Matrices)

```
let matrix: number[][] = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9],
];
console.log(matrix[1][2]); // 6
```

5.7 Practical Patterns

```
// Sum
const nums = [1, 2, 3, 4, 5];
const sum = nums.reduce((acc, n) => acc + n, 0);

// Filter + map (prefer functional array methods over manual loops for readability &
const evens = nums.filter(n => n % 2 === 0);
const doubled = nums.map(n => n * 2);

// Destructuring a tuple
const student: [number, string, number] = [101, "Mahesh", 95];
const [id, name, marks] = student;
```

Best practice: prefer `.map`/`.filter`/`.reduce` over manual `for` loops for transformations — they are declarative, immutable by default (they return new arrays), and compose well. Reach for `for...of` only when you need early `break`/`continue` or async iteration.

6. Objects & Type Aliases

6.1 Object Types

```
let user: { id: number; name: string; email: string; isActive: boolean } = {
  id: 1,
  name: "Mahesh",
  email: "mahesh@example.com",
  isActive: true,
};
```

6.2 Nested Objects

```
let employee: {
  id: number;
  name: string;
  address: { street: string; city: string; pin: number };
} = {
  id: 101,
  name: "Sita",
  address: { street: "Road No. 12", city: "Hyderabad", pin: 500081 },
};
```

For anything beyond a trivial one-off shape, extract nested object shapes into their own named `type` / `interface` rather than inlining — this avoids duplication if the nested shape is reused, and produces far more readable compiler error messages.

6.3 Optional Properties

```
type Product = {
  id: number;
  name: string;
  price: number;
  discount?: number; // may or may not be present
};
```

6.4 Readonly Properties

```
type User = {
  readonly id: number;
  name: string;
  readonly email: string;
};

const user: User = { id: 1, name: "Mahesh", email: "mahesh@example.com" };
user.name = "Samsani"; // Correct — allowed
user.id = 2;           // Incorrect — id is readonly
```

`readonly` is a **compile-time-only** guarantee — it does not freeze the object at runtime (use `Object.freeze()` for that). It prevents *accidental reassignment in your own codebase* but does not stop a consumer from mutating the object via `as` casting or plain JS.

6.5 The `type` Keyword (Type Aliases)

```
type User = {  
  id: number;  
  name: string;  
  email: string;  
  isActive: boolean;  
};  
  
const u1: User = { id: 1, name: "Mahesh", email: "mahesh@example.com", isActive: true };  
const users: User[] = [u1];
```

Type aliases centralize a shape definition so it can be reused everywhere (**DRY**), and — critically — when the shape changes, every usage site is re-validated by the compiler automatically.

6.6 Index Signatures

For objects whose keys aren't known ahead of time (e.g., a dictionary/map):

```
type ScoreBoard = {  
  [playerName: string]: number;  
};  
  
const scores: ScoreBoard = { alice: 95, bob: 88 };
```

With `noUncheckedIndexedAccess` enabled (recommended — see §2.3), `scores["carol"]` is correctly typed as `number | undefined`, forcing you to handle the "key might not exist" case rather than assuming it's always a `number`.

6.7 `satisfies` — Validate Without Widening (TS 4.9+)

A common production pain point: annotating a literal with a type loses the literal's specific inferred type (widened to the annotation), while *not* annotating loses validation. `satisfies` gives you both:

```
type RGB = [number, number, number];  
  
const palette = {
```



```
    red: [255, 0, 0],
    green: "#00ff00", // Incorrect – TS error caught immediately, even though palette
  } satisfies Record<string, RGB | string>;

// palette.red is still inferred as [number, number, number], not widened to RGB | s
```

`satisfies` is now the idiomatic way to validate configuration objects, route tables, and literal maps without sacrificing precise inference — prefer it over a plain type annotation whenever you want both validation *and* literal inference.

7. Interfaces

7.1 Interface Basics

Interfaces define the **structure/contract** of an object. TypeScript uses **structural typing** ("duck typing"): any object matching the shape satisfies the interface, regardless of how it was declared.

```
interface User {
  id: number;
  name: string;
  email: string;
  isActive: boolean;
}

let user: User = { id: 1, name: "Mahesh", email: "mahesh@example.com", isActive: true }
```

7.2 Extending Interfaces

```
interface Person {
  id: number;
  name: string;
}

interface Employee extends Person {
  department: string;
  salary: number;
}

let emp: Employee = { id: 101, name: "Sita", department: "QA", salary: 75000 };
```

An interface can extend multiple interfaces: `interface A extends B, C { ... }.`

7.3 Optional & Readonly Members

```
interface Product {
  id: number;
  name: string;
  price: number;
  discount?: number;    // optional
  description?: string;
}

interface Config {
  readonly apiUrl: string;
  readonly timeout: number;
  retries: number;
}
```

7.4 Interfaces for Function & Callable Shapes

```
interface Comparator<T> {
  (a: T, b: T): number; // call signature
}

const byAge: Comparator<{ age: number }> = (a, b) => a.age - b.age;
```

7.5 Declaration Merging (Unique to Interfaces)

```
interface Window {
  myCustomProperty: string;
}

// This second declaration MERGES with the first — both members exist on Window.
interface Window {
  anotherProperty: number;
}
```

Declaration merging is why interfaces are the standard choice for **augmenting third-party or global types** (e.g., extending Express's `Request` object with a custom `user` property in middleware). `type` aliases cannot do this — attempting to declare the same type alias twice is a compile error.

7.6 Interface vs Type Alias — The Real Decision Matrix

Feature	interface	type
Declaring object/class shapes	Yes — Idiomatic	Yes — Works

Feature	interface	type
Extending	<code>extends</code> (clear error messages)	<code>&</code> intersection (can produce <code>never</code> on conflicts, harder to debug)
Declaration merging	Yes — Supported	No — Not supported
Unions, tuples, primitives, mapped/conditional types	No — Not supported	Yes — Only <code>type</code> can do this
Implements (by classes)	Yes — Natural fit	Yes — Also works
Performance on huge object shapes	Generally faster to check (named, cached)	Can be slower for deeply nested unions

Senior-level rule of thumb: use `interface` for anything that describes the shape of an object or a class contract (especially public API surfaces, since they support declaration merging for extension by consumers). Use `type` for unions, tuples, function types, mapped/conditional types, and anything that is fundamentally not "an object with named properties." Many production style guides (e.g., large-scale Google/Microsoft-influenced codebases) mandate `interface` for object shapes and `type` for everything else — consistency matters more than which specific rule you pick.

7.7 Interfaces and Classes — `implements`

```
interface Shape {
  area(): number;
  perimeter(): number;
}

class Rectangle implements Shape {
  constructor(private width: number, private height: number) {}
  area(): number { return this.width * this.height; }
  perimeter(): number { return 2 * (this.width + this.height); }
}
```

`implements` is a compile-time-only check that a class satisfies an interface's shape — it does not copy any implementation (unlike `extends`), and a class can `implements` many interfaces but only `extends` one class.

8. Enums & Literal Types

8.1 Numeric Enums

```
enum Direction {  
  Up,      // 0  
  Down,    // 1  
  Left,    // 2  
  Right,   // 3  
}  
  
let dir: Direction = Direction.Left;  
console.log(dir); // 2
```

Numeric enums are **bidirectional** at runtime — `Direction[2]` returns `"Left"` — which is convenient for debugging but means the compiled JS includes a reverse-mapping object, adding bundle size.

8.2 String Enums

```
enum Status {  
  Success = "SUCCESS",  
  Failure = "FAILURE",  
  Pending = "PENDING",  
}  
  
let s: Status = Status.Success;  
console.log(s); // "SUCCESS"
```

String enums are **not** bidirectionally mapped, are more debuggable (`console.log` shows a meaningful string, not a number), and are generally preferred over numeric enums in production code for exactly this reason.

8.3 `const enum` (Zero Runtime Cost)

```
const enum LogLevel { Debug, Info, Warn, Error }  
const level = LogLevel.Warn; // inlined to `2` at compile time — no LogLevel object
```

`const enum` is fully inlined and erased, producing zero runtime footprint — but it is **incompatible with** `isolatedModules` (required by esbuild/swc/Babel single-file transpilation) because those tools can't see across files to know what to inline. Most modern production toolchains therefore avoid `const enum`.

8.4 The Modern Alternative: Union of String Literals + `as const` Object

Because of the caveats above, many senior TypeScript engineers now prefer this pattern over `enum` entirely:

```
const Status = {
  Success: "SUCCESS",
  Failure: "FAILURE",
  Pending: "PENDING",
} as const;

type Status = (typeof Status)[keyof typeof Status]; // "SUCCESS" | "FAILURE" | "PENDING"

function handle(status: Status) { /* ... */ }
handle(Status.Success); // Correct
handle("SUCCESS");      // Correct — plain string literals are also valid, unlike re
```

Why this is often preferred: it has zero runtime overhead beyond a plain object, is fully tree-shakeable, plays perfectly with `isolatedModules`, and — unlike TS `enum` — a plain string literal is directly assignable without importing the enum, which matters a lot when the value crosses a network boundary (e.g., JSON from an API) since `enum` values received as raw strings require an unsafe cast, whereas string literal unions accept them naturally.

8.5 Literal Types

```
let role: "admin" = "admin";
role = "admin"; // Correct
// role = "user"; // Incorrect — only the literal "admin" is allowed
```

8.6 Union of Literals

```
type DirectionType = "up" | "down" | "left" | "right";
let move: DirectionType = "up";
move = "left"; // Correct
// move = "jump"; // Incorrect — Error
```

Union literal types are the backbone of **discriminated unions** (§9.3) — one of the single most powerful patterns in production TypeScript for modeling state machines, API responses, and Redux-style actions.

9. Union, Intersection & Discriminated Unions

9.1 Union Types (|)

A variable typed as a union can hold a value from any of the listed types.

```
let id: number | string;
id = 101;           // Correct
id = "QA-101";     // Correct
// id = true;      // Incorrect – Error
```

9.2 Intersection Types (&)

Combines multiple types into one type that has **all** properties from **all** constituent types.

```
type Person = { name: string; age: number };
type Employee = { empId: number; department: string };

type EmployeeDetails = Person & Employee;

const e: EmployeeDetails = { name: "Sita", age: 30, empId: 101, department: "QA" };
```

9.3 Discriminated (Tagged) Unions — The Most Important Pattern in This Section

A discriminated union is a union of object types that share a common **literal-typed property** (the "discriminant" or "tag"). This is *the* idiomatic way to model states/variants safely in production TypeScript.

```
type SuccessResponse = { status: "success"; data: { id: number; name: string } };
type ErrorResponse = { status: "error"; message: string };
type LoadingResponse = { status: "loading" };

type ApiResponse = SuccessResponse | ErrorResponse | LoadingResponse;

function render(response: ApiResponse): string {
  switch (response.status) {
    case "success":
      return `Loaded: ${response.data.name}`; // TS knows `.data` exists here
    case "error":
      return `Error: ${response.message}`;    // TS knows `.message` exists here
    case "loading":
      return "Loading...";
    default:
      // Exhaustiveness check: if a new variant is added to ApiResponse and this
```

```
// switch isn't updated, `response` here is NOT `never`, and TS raises a
// compile error at this line – catching incomplete handling at build time.
const _exhaustive: never = response;
throw new Error(`Unhandled response: ${JSON.stringify(_exhaustive)}`);
}
}
```

This exhaustiveness-check pattern (`const _exhaustive: never = x`) is a hallmark of senior TypeScript code. It converts "did the developer forget to handle a new case?" from a runtime bug discovered in production into a compile-time build failure — the type system enforces completeness for you.

9.4 Type Narrowing (Preview — full coverage in §10)

```
function printId(id: number | string) {
  if (typeof id === "string") {
    console.log("String ID: " + id.toUpperCase());
  } else {
    console.log("Number ID: " + id.toFixed(2));
  }
}
```

9.5 Real-World Composition Example

```
type User = { id: number; name: string };
type WithTimestamps = { createdAt: Date; updatedAt: Date };
type WithRoles = { roles: string[]; isActive: boolean };

type FullUserRecord = User & WithTimestamps & WithRoles;
```

Guideline: use unions to express "one of several possible shapes" (variance), and intersections to express "compose several independent concerns into one shape" (composition). Reaching for intersection types to build up entities from small, single-purpose fragments is itself an application of **composition over inheritance** — see §17.

10. Type Narrowing, Assertions & Type Guards

10.1 The `as` Keyword (Type Assertion)

```
let value: unknown = "Hello World";
let strLen: number = (value as string).length;
```

`as` tells the compiler "trust me, treat this value as type X" — **it performs no runtime check or conversion whatsoever**. Use it only when you have external knowledge the compiler cannot infer (e.g., you know a DOM query will return a specific element type). Misusing `as` to silence errors is functionally equivalent to using `any` and is a common source of production bugs — prefer type guards wherever possible.

```
const el = document.getElementById("app") as HTMLDivElement; // legitimate use
```

The double-assertion escape hatch `value as unknown as TargetType` exists for genuinely incompatible casts, but its presence in a PR should always prompt a reviewer to ask "why can't this be proven safely?"

10.2 `instanceof` Operator

```
class Dog { bark() { console.log("Woof"); } }
class Cat { meow() { console.log("Meow"); } }

let pet: Dog | Cat = new Dog();

if (pet instanceof Dog) {
  pet.bark(); // Correct – narrowed to Dog
}
```

`instanceof` works because **classes** (unlike interfaces/types) compile to real JS constructor functions that exist at runtime.

10.3 `typeof` Operator (Primitives)

```
function process(val: string | number) {
  if (typeof val === "string") {
    console.log(val.toUpperCase());
  } else {
    console.log(val.toFixed(2));
  }
}
```

`typeof` narrowing works for: `"string"`, `"number"`, `"boolean"`, `"symbol"`, `"undefined"`, `"function"`, `"bigint"`, `"object"`.

10.4 `in` Operator (Property Existence)

```
type Admin = { name: string; role: string };
type RegularUser = { name: string; email: string };
```



```
function describe(person: Admin | RegularUser) {
  if ("role" in person) {
    console.log(person.role); // narrowed to Admin
  } else {
    console.log(person.email); // narrowed to RegularUser
  }
}
```

10.5 Custom Type Guards (User-Defined Type Predicates)

```
type Circle = { kind: "circle"; radius: number };
type Square = { kind: "square"; side: number };
type Shape = Circle | Square;

function isCircle(shape: Shape): shape is Circle {
  return shape.kind === "circle";
}

function getArea(shape: Shape): number {
  if (isCircle(shape)) {
    return Math.PI * shape.radius ** 2;
  }
  return shape.side * shape.side;
}
```

The `shape is Circle` return type is a **type predicate** — it tells the compiler "if this function returns `true`, narrow the argument to `Circle` in the calling scope." Custom guards are essential once narrowing logic is reused across multiple call sites (DRY) or is too complex for a single `typeof`/`in` check.

10.6 Assertion Functions (`asserts`)

A related but distinct pattern — instead of returning a boolean, an assertion function **throws** if the condition is false, and TypeScript narrows the type for all code *after* the call:

```
function assertIsString(val: unknown): asserts val is string {
  if (typeof val !== "string") {
    throw new Error("Expected a string");
  }
}

function process(input: unknown) {
  assertIsString(input);
  console.log(input.toUpperCase()); // narrowed to string here, no `if` needed
}
```

This pattern is common in production **guard-clause / fail-fast validation** at function entry points.

10.7 Non-Null Assertion (!) — Use Sparingly

```
function getLength(s: string | null) {  
    return s!.length; // "I promise this is not null" – no runtime check performed  
}
```

! is one of the "Common Mistakes" flagged by senior reviewers when overused (see §22) — it silences the compiler exactly where it's trying to protect you from a real runtime crash. Prefer an explicit check (`if (s === null) throw ...`) or the nullish coalescing operator (`s ?? defaultValue`) instead.

10.8 Best-Practice Ordering of Narrowing Techniques

1. **Prefer discriminated unions + `switch`** (compiler-enforced exhaustiveness) whenever you control the shape.
2. **Prefer built-in guards** (`typeof`, `instanceof`, `in`, `Array.isArray`) for primitives/classes/known-shape checks.
3. **Write a custom type predicate** when guard logic is reused or non-trivial.
4. **Use assertion functions** at validation/guard-clause boundaries (start of a function, API handler).
5. **Reach for `as / !` only as a last resort**, and always leave a comment explaining *why* the compiler can't prove it.

11. Classes & Object-Oriented Programming

11.1 Class Basics

```
class Person {  
    name: string;  
    age: number;  
    constructor(name: string, age: number) {  
        this.name = name;  
        this.age = age;  
    }  
}  
  
const p = new Person("Mahesh", 25);
```

11.2 Constructor Property Shorthand (Idiomatic TS)

Instead of declaring properties and then re-assigning them in the constructor body, TypeScript lets you declare and assign in one step by prefixing constructor parameters with an access modifier:

```
class Person {
  constructor(
    public name: string,
    private age: number,
    readonly id: number = Date.now(),
  ) {}
}
```

This is the standard idiom in production TS — verbose manual property declarations are considered a code smell once you know this shorthand.

11.3 Properties: Instance vs Static

```
class Counter {
  count: number = 0;           // instance property – each object gets its own co
  static totalInstances = 0;    // static property – belongs to the class itself,

  constructor() {
    Counter.totalInstances++;
  }
}
```

11.4 Methods

```
class Person {
  constructor(private name: string) {}
  greet(): string {
    return `Hello, I am ${this.name}`;
  }
}
```

11.5 Access Modifiers

Modifier	Accessible From	Notes
<code>public</code> (default)	Everywhere	Implicit if omitted
<code>private</code>	Only within the declaring class	TS compile-time only — see caveat below

Modifier	Accessible From	Notes
<code>protected</code>	The declaring class and its subclasses	Enables controlled extension points
<code>#trueprivate</code> (JS native)	Only within the declaring class — enforced at runtime	True encapsulation, unlike <code>private</code>

Critical caveat: TypeScript's `private`/`protected` are erased at compile time — the emitted JavaScript has no actual access restriction, so reflection, `JSON.stringify`, or a runtime `as any` cast can still reach "private" members. For genuine runtime-enforced encapsulation (e.g., a security-sensitive value), use native **ECMAScript private fields** (`#balance`) instead:

```
class BankAccount {
  #balance: number = 0; // truly inaccessible outside this class, even at runtime
  deposit(amount: number) { this.#balance += amount; }
  get balance() { return this.#balance; }
}
```

11.6 Getters & Setters (Encapsulation)

```
class Temperature {
  #celsius = 0;
  get fahrenheit(): number { return this.#celsius * 9 / 5 + 32; }
  set fahrenheit(value: number) { this.#celsius = (value - 32) * 5 / 9; }
}
```

Getters/setters let you expose a clean property-like API while retaining the ability to validate, compute, or log on access — a key encapsulation technique.

11.7 Static Members

```
class MathUtil {
  static readonly PI: number = 3.14159;
  static area(radius: number): number {
    return MathUtil.PI * radius ** 2;
  }
}
console.log(MathUtil.area(5)); // 78.53975
```

Use `static` for utility/factory functions and constants that are conceptually tied to a class but don't need per-instance state. Overuse of static methods, however, is a

common anti-pattern — it prevents dependency injection and mocking in tests (see §19), so reserve it for genuinely stateless pure-function helpers.

11.8 Abstract Classes

```
abstract class Shape {
  constructor(public readonly name: string) {}
  abstract area(): number; // must be implemented by subclasses – no body allowed here

  describe(): string { // concrete method – shared by all subclasses
    return `${this.name} has area ${this.area().toFixed(2)}`;
  }
}

class Circle extends Shape {
  constructor(private radius: number) { super("Circle"); }
  area(): number { return Math.PI * this.radius ** 2; }
}

// const s = new Shape("Generic"); // Incorrect – cannot instantiate an abstract class
```

Abstract classes are the right tool when subclasses share **both** a contract (methods that must exist) **and** concrete shared implementation (methods with a body). If there's no shared implementation, prefer a plain `interface` — it's more flexible (multiple implementation, no forced hierarchy).

11.9 Inheritance

```
class Animal {
  constructor(public name: string) {}
  speak(): void { console.log(`${this.name} makes a sound`); }
}

class Dog extends Animal {
  constructor(name: string, private breed: string) {
    super(name); // must call super() before using `this` in a derived constructor
  }
  override speak(): void { // `override` keyword (with noImplicitOverride) catches typos
    console.log(`${this.name} barks`);
  }
}

const d = new Dog("Buddy", "Labrador");
d.speak(); // "Buddy barks"
```

11.10 Polymorphism

```
const animals: Animal[] = [new Animal("Generic"), new Dog("Buddy", "Labrador")];
animals.forEach(a => a.speak()); // each object's own `speak()` is invoked – runtime
```

11.11 Composition Over Inheritance (Senior-Level Guidance)

Deep inheritance hierarchies are notoriously fragile ("the fragile base class problem" — a change in a base class can silently break many descendants). Production-grade TypeScript strongly favors **composing behavior from small, focused, injected collaborators** over building tall `class C extends B extends A` chains:

```
// Incorrect – Inheritance-heavy – brittle, hard to test in isolation
class Logger {}
class AuditableService extends Logger {}
class UserService extends AuditableService {}

// Correct – Composition – flexible, each dependency independently testable/mockable
interface Logger { log(msg: string): void; }
interface AuditTrail { record(action: string): void; }

class UserService {
  constructor(
    private readonly logger: Logger,
    private readonly audit: AuditTrail,
  ) {}

  createUser(name: string) {
    this.logger.log(`Creating user ${name}`);
    this.audit.record(`create-user:${name}`);
  }
}
```

This is precisely the **Dependency Inversion Principle** (§17.5) and the standard pattern behind dependency-injection frameworks (NestJS, InversifyJS, Angular DI).

11.12 Interfaces + Classes: Design by Contract

```
interface Repository<T> {
  findById(id: string): Promise<T | null>;
  save(entity: T): Promise<void>;
}

class InMemoryUserRepository implements Repository<User> {
  private store = new Map<string, User>();
}
```

```
async findById(id: string): Promise<User | null> { return this.store.get(id) ?? null; }
async save(user: User): Promise<void> { this.store.set(user.id, user); }
}
```

Programming against an `interface` (the **Repository pattern**, §16.6) rather than a concrete class allows the implementation (in-memory, SQL, or a mock for tests) to be swapped without touching consumer code — this is the core value of interfaces in OOP TypeScript.

12. Generics — Basic to Advanced

12.1 Why Generics?

```
// Incorrect — Without generics — duplicated logic per type (violates DRY)
function addNumbers(a: number, b: number): number { return a + b; }
function addStrings(a: string, b: string): string { return (a as any) + b; }

// Correct — With generics — one reusable, fully type-safe implementation
function identity<T>(value: T): T { return value; }

const n = identity<number>(42); // T inferred/explicit as number
const s = identity<string>("Hello"); // T inferred as string — explicit <string> not
```

Generics let you write algorithms and data structures that are **parameterized by type**, preserving full type safety without duplicating code per type — the type-system equivalent of DRY.

12.2 Generic Functions

```
function firstElement<T>(arr: T[]): T | undefined {
  return arr[0];
}
```

12.3 Generic Interfaces

```
interface ApiResponse<T> {
  data: T;
  success: boolean;
  message?: string;
}

const userRes: ApiResponse<{ id: number; name: string }> = {
```

```
data: { id: 1, name: "Mahesh" },
success: true,
message: "Fetched successfully",
};
```

12.4 Generic Classes

```
class Box<T> {
  private value: T;
  constructor(value: T) { this.value = value; }
  getValue(): T { return this.value; }
}

const numBox = new Box<number>(100);
const strBox = new Box<string>("TypeScript");
```

12.5 Constraints (`extends`)

```
function getLength<T extends { length: number }>(item: T): number {
  return item.length;
}

getLength("Hello");           // Correct - 5
getLength([1, 2, 3, 4]);      // Correct - 4
// getLength(123);           // Incorrect - number has no .length
```

12.6 Multiple Type Parameters & `keyof` Constraints

```
function getProperty<T, K extends keyof T>(obj: T, key: K): T[K] {
  return obj[key];
}

const user = { id: 1, name: "Mahesh" };
const name = getProperty(user, "name"); // inferred as string
// getProperty(user, "email");           // Incorrect - "email" is not a key of user
```

`K extends keyof T` is one of the highest-leverage generic patterns in production TS — it makes property-access helpers (getters, form-field accessors, ORM query builders) fully type-safe against typos.

12.7 Default Generic Parameters

```
interface PaginatedResult<T = unknown> {
  items: T[];
  page: number;
```



```
totalPages: number;
}
```

12.8 Generic Constraints With Conditional Defaults

```
type ApiResult<T, E = Error> =
  | { ok: true; value: T }
  | { ok: false; error: E };
```

12.9 Conditional Types

Conditional types let you branch a type based on a type-level condition, similar to a ternary but evaluated by the compiler:

```
type IsString<T> = T extends string ? "yes" : "no";
type A = IsString<string>; // "yes"
type B = IsString<number>; // "no"

// Practical example: unwrap a Promise's inner type
type Awaited2<T> = T extends Promise<infer U> ? U : T;
type Result = Awaited2<Promise<number>>; // number
```

The `infer` keyword introduces a new type variable to be inferred within the `extends` clause — this is how TypeScript's own built-in `ReturnType<T>`, `Parameters<T>`, and `Awaited<T>` utility types are implemented internally.

12.10 Mapped Types

Mapped types let you build a new object type by transforming every property of an existing type:

```
type Readonly2<T> = { readonly [K in keyof T]: T[K] };
type Partial2<T> = { [K in keyof T]?: T[K] };
type Nullable<T> = { [K in keyof T]: T[K] | null };

// Key remapping with `as` (TS 4.1+)
type Getters<T> = {
  [K in keyof T as `get${Capitalize<string & K>}`]: () => T[K];
};
interface Person { name: string; age: number; }
type PersonGetters = Getters<Person>;
// { getName: () => string; getAge: () => number }
```

12.11 Template Literal Types

Template literal types let you build string-literal unions programmatically at the type level — extremely valuable for typed routes, CSS-in-JS, i18n keys, and event names:

```
type HttpMethod = "GET" | "POST" | "PUT" | "DELETE";
type Endpoint = "/users" | "/orders";
type Route = `${HttpMethod} ${Endpoint}`;
// "GET /users" | "GET /orders" | "POST /users" | ... (8 combinations, generated aut

type EventName<T extends string> = `on${Capitalize<T>}`;
type ClickEvent = EventName<"click">; // "onClick"
```

12.12 Variance & Function Parameter Bivariance (Advanced Nuance)

TypeScript function types are checked contravariantly for parameters under `strictFunctionTypes` (part of `strict`). This matters when designing generic callback-accepting APIs — a function accepting a narrower parameter type is *not* automatically substitutable where a wider one is expected, which is the mathematically sound behavior and prevents an entire class of "safe-looking but unsound" callback bugs.

12.13 Production Guidance on Generics

- Name generic parameters meaningfully once there's more than one (`T`, `K`, `V` for trivial single-purpose generics; `TInput`, `TOutput`, `TEntity` for domain-specific ones in larger codebases).
- Add constraints (`extends`) as narrowly as needed — an unconstrained `T` in a function that clearly needs `.length` or `.id` is a missed opportunity for a better compiler error at the call site.
- Prefer generics over `any` in every "any" generic-container situation — repositories, caches, event emitters, HTTP client wrappers, state containers.
- Don't over-genericize: if a function is only ever called with one concrete type in the entire codebase, a generic adds indirection without benefit — YAGNI applies to type parameters too.

13. Modules, Namespaces & Project Structure

13.1 Export / Import (Named)

```
// math.ts
export const PI = 3.14159;
export function add(a: number, b: number): number { return a + b; }

// app.ts
import { PI, add } from "./math";
console.log(add(5, 10)); // 15
```

13.2 Default Export / Import

```
// user.ts
export default class User {
  constructor(public name: string) {}
  greet() { return `Hello, ${this.name}`; }
}

// app.ts
import User from "./user"; // any local name is valid – no curly braces
```

Production guidance: many senior-level style guides (and the TypeScript team itself, informally) now **prefer named exports over default exports**, because: (a) named exports are automatically refactor-safe (renaming the export renames every import via IDE tooling; default exports can be imported under any arbitrary name, defeating that), (b) named exports enable better tree-shaking analysis by bundlers, and (c) they support multiple exports per file naturally. Reserve default exports for framework-mandated conventions (e.g., a Next.js page file) where the tooling requires them.

13.3 Re-exports & Barrel Files

```
// utils/index.ts (a "barrel" file)
export { formatDate } from "./date";
export { slugify } from "./string";
export * from "./validation";
```

Barrel files simplify imports (`import { formatDate, slugify } from "@/utils"`) but at scale can hurt build performance and tree-shaking (bundlers sometimes can't eliminate unused re-exports) — many large monorepos deliberately avoid deep barrel chains for this reason. Use them for small, stable, public-facing module boundaries; avoid them for large internal utility folders.

13.4 Type-Only Imports/Exports

```
import type { User } from "./types"; // guarantees this import is erased entirely a
```



```
|   └─ utils/
|   └─ errors/
└─ config/
    └─ env.ts           # validated environment config (see §18.4)
└─ index.ts            # composition root / entry point
```

Feature-based ("vertical slice") organization — grouping by business domain rather than by technical layer (`controllers/`, `services/`, `models/` folders scattered separately) — is the pattern most large production codebases converge on, because it keeps everything related to one feature co-located, minimizing cross-folder churn when a feature changes and making it trivial to delete a feature cleanly.

14. Asynchronous Programming — Event Loop to Production Patterns

14.1 The JavaScript Event Loop (Foundational Mental Model)

JavaScript is single-threaded. Asynchronous operations (I/O, timers, network) don't block that thread — they're handed off to the runtime (Node/browser APIs), and their callbacks are queued to run later:

- **Call stack** — currently executing synchronous code.
- **Microtask queue** — Promise `.then/` `.catch/` `.finally` callbacks and `queueMicrotask`. Drained *completely* before the next macrotask.
- **Macrotask queue** — `setTimeout`, `setInterval`, I/O callbacks, `setImmediate` (Node).

Understanding that **microtasks always run before the next macrotask** explains many "why did this log before that" surprises and is essential when reasoning about ordering in concurrent async code.

14.2 Promises

```
const promise = new Promise<string>((resolve, reject) => {
  const success = true;
  if (success) resolve("Operation successful!");
  else reject(new Error("Something went wrong"));
});

promise.then(data => console.log(data)).catch(err => console.error(err));
```

Always `reject` with an `Error` instance (or subclass), never a plain string — this preserves a stack trace and lets `catch` blocks reliably use `instanceof Error`.

14.3 `async` / `await`

```
async function getData(): Promise<string> {  
  return "Hello from async function";  
}  
getData().then(data => console.log(data));
```

An `async` function **always** returns a `Promise`, even if the body returns a plain value (it's implicitly wrapped) — this is a common early misconception worth internalizing.

```
async function loadMessage() {  
  console.log("Fetching message...");  
  const msg = await getData(); // pauses here until the Promise settles, without blo  
  console.log(msg);  
}
```

14.4 Error Handling in Async Code

```
async function getUser(id: number): Promise<User> {  
  try {  
    const res = await fetch(`https://api.example.com/users/${id}`);  
    if (!res.ok) throw new Error(`HTTP ${res.status}`);  
    const user: unknown = await res.json();  
    return validateUser(user); // see §18.4 – never trust `unknown` JSON without val  
  } catch (error) {  
    // `error` in a catch clause is typed `unknown` under `strict`/`useUnknownInCatc  
    // you must narrow before using it.  
    if (error instanceof Error) {  
      console.error("Error:", error.message);  
    }  
    throw error; // re-throw unless this function is specifically meant to swallow i  
  } finally {  
    console.log("Request completed");  
  }  
}
```

14.5 Promise Combinators — Choosing the Right One

Combinator	Behavior	Use When
<code>Promise.all</code>	Waits for all; rejects immediately if any one rejects	All results are required; failure of one invalidates the whole batch
<code>Promise.allSettled</code>	Waits for all; never rejects , returns per-item status	You need every result regardless of individual failures (e.g., batch notifications)
<code>Promise.race</code>	Settles as soon as the first promise settles (resolve or reject)	Timeouts, "fastest wins" scenarios
<code>Promise.any</code>	Settles as soon as the first one resolves; rejects only if <i>all</i> reject	Trying multiple redundant sources, first success wins

```
// Fetch multiple resources concurrently instead of sequentially – a very common per
const [users, orders] = await Promise.all([fetchUsers(), fetchOrders()]);

// Timeout pattern using Promise.race
function withTimeout<T>(promise: Promise<T>, ms: number): Promise<T> {
  const timeout = new Promise<never>((_, reject) =>
    setTimeout(() => reject(new Error("Timed out")), ms)
  );
  return Promise.race([promise, timeout]);
}
```

14.6 Sequential vs Concurrent `await` — A Common Performance Bug

```
// Incorrect – Sequential – each await blocks the next, total time = sum of all requ
const user = await fetchUser(id);
const orders = await fetchOrders(id);

// Correct – Concurrent – both requests fire immediately, total time = the slower of
const [user2, orders2] = await Promise.all([fetchUser(id), fetchOrders(id)]);
```

Only await sequentially when a later call genuinely depends on an earlier call's result — otherwise, kick off independent async work concurrently.

14.7 Async Iteration

```
async function* paginate(url: string): AsyncGenerator<Item[]> {
  let next: string | null = url;
  while (next) {
    const res = await fetch(next);
    const page = await res.json();
    yield page.items;
    next = page.nextPageUrl;
  }
}

for await (const items of paginate("/api/items")) {
  process(items);
}
```

14.8 Fetch API Example With Typed Response

```
interface ApiUser { id: number; name: string; email: string; }

async function fetchUsers(): Promise<ApiUser[]> {
  try {
    const res = await fetch("https://jsonplaceholder.typicode.com/users");
    if (!res.ok) throw new Error(`HTTP error! status: ${res.status}`);
    const users: unknown = await res.json();
    if (!Array.isArray(users)) throw new Error("Unexpected response shape");
    return users as ApiUser[]; // in production, validate with a schema library – see
  } catch (error) {
    console.error("Failed to fetch users:", (error as Error).message);
    throw error;
  }
}
```

14.9 Cancellation With `AbortController`

```
async function fetchWithCancel(url: string, signal: AbortSignal) {
  const res = await fetch(url, { signal });
  return res.json();
}

const controller = new AbortController();
setTimeout(() => controller.abort(), 5000); // cancel if it takes too long
fetchWithCancel("/api/slow", controller.signal);
```

Production async code — especially in UI code where a component may unmount, or backend code with client-disconnect handling — should propagate `AbortSignal`

through call chains rather than letting orphaned requests run to completion wastefully.

14.10 Best Practices Summary

- Always handle errors with `try/catch` around every `await`-ing async function boundary — an unhandled rejection crashes Node processes (`unhandledRejection`) and silently fails in browsers.
- Avoid "callback hell" — `async/await` on top of Promises is strictly preferred over raw `.then()` chains for anything beyond a single step.
- Never mix `async/await` with `.then()` chains in the same function — pick one style per function for readability.
- Use `Promise.all/allSettled` to parallelize independent I/O — sequential `await` is a top production performance anti-pattern.
- Keep async functions small and focused — a 200-line `async function` handling five different concerns is a Single-Responsibility-Principle violation just like any other oversized function.

15. Advanced/Utility Types & the Type-Level Programming Toolkit

15.1 `keyof`

```
type User = { id: number; name: string; email: string };
type UserKeys = keyof User; // "id" | "name" | "email"
const key: UserKeys = "id"; // Correct
```

15.2 `typeof` (Type Query, not the runtime operator)

```
const user = { id: 1, name: "Mahesh" };
type UserType = typeof user; // { id: number; name: string }
```

Combining `typeof` + `keyof` lets you derive types directly from runtime values/config objects — extremely useful for deriving a type from a single source of truth (DRY at the type level):

```
const routes = { home: "/", about: "/about", contact: "/contact" } as const;
type RouteKey = keyof typeof routes; // "home" | "about" | "contact"
type RoutePath = (typeof routes)[RouteKey]; // "/" | "/about" | "/contact"
```

15.3 Built-In Utility Types — Complete Reference

```
type User = { id: number; name: string; email: string; age: number };

type PartialUser    = Partial<User>;           // all properties optional
type RequiredUser   = Required<PartialUser>;    // all properties mandatory
type ReadonlyUser   = Readonly<User>;          // all properties readonly
type Picked         = Pick<User, "id" | "name">; // only id and name
type Omitted        = Omit<User, "email">;      // everything except email
type ScoreMap       = Record<string, number>;   // dictionary type: keys string,
type MaybeUser      = User | null;
type NonNull        = NonNullable<MaybeUser>; // strips null/undefined -> User
```

```
// Record – dictionaries/maps
type ScoreMap = Record<string, number>;
const scores: ScoreMap = { math: 95, science: 88, english: 92 };

// Pick – select a subset
type UserPreview = Pick<User, "id" | "name">;

// Omit – remove specific keys
type UserWithoutEmail = Omit<User, "email">;
```

15.4 Function-Related Utility Types

```
function createUser(name: string, age: number): { id: number; name: string } {
  return { id: Date.now(), name };
}

type CreateUserParams = Parameters<typeof createUser>; // [name: string, age: number]
type CreateUserReturn  = ReturnType<typeof createUser>; // { id: number; name: string }
type CreateUserAsync   = Awaited<ReturnType<typeof getUserAsync>>; // unwraps a Promise
```

`ReturnType` / `Parameters` are heavily used to derive types from existing functions rather than redeclaring them — another DRY application, and crucial when wrapping/decorating third-party functions.

15.5 String Manipulation Utility Types

```
type Shout = Uppercase<"hello">; // "HELLO"
type Whisper = Lowercase<"HELLO">; // "hello"
type Title = Capitalize<"typescript">; // "Typescript"
type Uncap = Uncapitalize<"TypeScript">; // "typeScript"
```

15.6 Exclude / Extract

```
type AllRoles = "admin" | "editor" | "viewer" | "guest";
type PrivilegedRoles = Exclude<AllRoles, "viewer" | "guest">; // "admin" | "editor"
type ReadOnlyRoles = Extract<AllRoles, "viewer" | "guest">; // "viewer" | "guest"
```

15.7 Deep Utility Types (Custom, Because TS Doesn't Ship Them Built-In)

Production codebases frequently need `DeepPartial` / `DeepReadonly` beyond the shallow built-ins — these are commonly hand-rolled with recursive mapped + conditional types:

```
type DeepPartial<T> = T extends object
  ? { [K in keyof T]?: DeepPartial<T[K]> }
  : T;

type DeepReadonly<T> = T extends object
  ? { readonly [K in keyof T]: DeepReadonly<T[K]> }
  : T;
```

15.8 Branded / Nominal Types

TypeScript's structural typing means `type UserId = string` and `type OrderId = string` are interchangeable — a common source of bugs when IDs of different entities get swapped. **Branding** simulates nominal typing to prevent this:

```
type UserId = string & { readonly __brand: "UserId" };
type OrderId = string & { readonly __brand: "OrderId" };

function getUser(id: UserId) { /* ... */ }

function toUserId(id: string): UserId { return id as UserId; } // one controlled cas

const uid = toUserId("u-123");
// getUser("u-123"); // Incorrect — a raw string is not a UserId
getUser(uid);        // Correct
```

This pattern is standard in large financial/e-commerce codebases where mixing up an ID type is a real, costly bug class.

15.9 Required, Readonly, and Combining Utility Types

```
type CompleteConfig = Required<ReadOnly<Partial<AppConfig>>>;
```

Utility types compose freely — chaining them is idiomatic and preferred over hand-writing an equivalent mapped type from scratch (DRY, and it leans on well-tested compiler-provided primitives).

15.10 Production Guidance

- Prefer built-in utility types over custom equivalents whenever one exists — they're battle-tested and instantly recognizable to any TS developer.
- Derive types from a single source of truth (`typeof someConstObject`, `ReturnType<typeof someFn>`) rather than maintaining two parallel, potentially-drifting definitions.
- Keep custom deep/recursive utility types in one shared `types/utils.ts` file — don't redefine `DeepPartial` in five different modules.

16. Design Patterns in TypeScript

Design patterns are reusable solutions to recurring design problems. TypeScript's type system (interfaces, generics, access modifiers) makes classic Gang-of-Four patterns considerably safer and more expressive than in plain JavaScript.

16.1 Singleton

Ensures a class has exactly one instance, with a global access point. Use sparingly — it's effectively global mutable state and hinders testability.

```
class ConfigManager {
  private static instance: ConfigManager;
  private constructor(private readonly settings: Record<string, string>) {}

  static getInstance(): ConfigManager {
    if (!ConfigManager.instance) {
      ConfigManager.instance = new ConfigManager(loadSettingsFromEnv());
    }
    return ConfigManager.instance;
  }
}
```

Production note: in most Node.js backends, module-level `const` exports already behave like singletons (a module is only ever evaluated once), making the classic

Singleton class often unnecessary boilerplate — prefer a simple exported instance unless you specifically need lazy initialization or subclassing.

16.2 Factory

Encapsulates object-creation logic, decoupling callers from concrete classes.

```
interface PaymentProcessor { charge(amount: number): Promise<void>; }
class StripeProcessor implements PaymentProcessor { async charge(amount: number) { /
class PaypalProcessor implements PaymentProcessor { async charge(amount: number) { /

type ProcessorType = "stripe" | "paypal";

function createPaymentProcessor(type: ProcessorType): PaymentProcessor {
  switch (type) {
    case "stripe": return new StripeProcessor();
    case "paypal": return new PaypalProcessor();
  }
}
```

16.3 Strategy

Encapsulates interchangeable algorithms behind a common interface, selected at runtime.

```
interface DiscountStrategy { apply(price: number): number; }
class NoDiscount implements DiscountStrategy { apply(price: number) { return price; }
class PercentageDiscount implements DiscountStrategy {
  constructor(private readonly percent: number) {}
  apply(price: number) { return price * (1 - this.percent / 100); }
}

class Checkout {
  constructor(private strategy: DiscountStrategy) {}
  total(price: number) { return this.strategy.apply(price); }
}
```

16.4 Observer

Defines a one-to-many dependency so that when one object changes state, all dependents are notified — the foundation of event systems, reactive state, and pub/sub.

```
type Listener<T> = (payload: T) => void;
```

```

class EventEmitter<T> {
  private listeners: Listener<T>[] = [];
  subscribe(listener: Listener<T>): () => void {
    this.listeners.push(listener);
    return () => { this.listeners = this.listeners.filter(l => l !== listener); }; /
  }
  emit(payload: T): void {
    for (const listener of this.listeners) listener(payload);
  }
}

const onUserCreated = new EventEmitter<{ id: number; name: string }>();
const unsubscribe = onUserCreated.subscribe(user => console.log(`Welcome ${user.name}

```

16.5 Decorator (Structural Pattern, Distinct From TS's `@decorator` Syntax)

Adds behavior to an object dynamically without altering other objects of the same class.

```

interface Coffee { cost(): number; description(): string; }

class SimpleCoffee implements Coffee {
  cost() { return 2; }
  description() { return "Coffee"; }
}

class MilkDecorator implements Coffee {
  constructor(private coffee: Coffee) {}
  cost() { return this.coffee.cost() + 0.5; }
  description() { return `${this.coffee.description()} + Milk`; }
}

const order = new MilkDecorator(new SimpleCoffee());

```

TypeScript also has native `@decorator` syntax (Stage-3 ECMAScript decorators, used heavily in Angular/NestJS) for annotating classes/methods declaratively — a related but syntactically distinct concept from the GoF Decorator pattern shown above.

16.6 Repository

Abstracts data-access logic behind a domain-focused interface, decoupling business logic from the persistence mechanism (SQL, NoSQL, in-memory, HTTP).

```
interface UserRepository {
  findById(id: string): Promise<User | null>;
  save(user: User): Promise<void>;
  delete(id: string): Promise<void>;
}

class PostgresUserRepository implements UserRepository {
  constructor(private readonly db: DatabaseClient) {}
  async findById(id: string) { return this.db.query<User>("SELECT * FROM users WHERE id = ?"); }
  async save(user: User) { /* ... */ }
  async delete(id: string) { /* ... */ }
}
```

16.7 Adapter

Converts the interface of one class into another interface clients expect — essential when integrating a third-party library whose API shape doesn't match your domain's contracts.

```
interface NotificationSender { send(to: string, message: string): Promise<void>; }

class TwilioSdkAdapter implements NotificationSender {
  constructor(private readonly twilioClient: ThirdPartyTwilioClient) {}
  async send(to: string, message: string) {
    await this.twilioClient.messages.create({ to, body: message }); // adapts SDK's
  }
}
```

16.8 Builder

Constructs complex objects step-by-step, useful when an object has many optional configuration fields.

```
class RequestBuilder {
  private url = "";
  private method: "GET" | "POST" = "GET";
  private headers: Record<string, string> = {};

  setUrl(url: string): this { this.url = url; return this; }
  setMethod(method: "GET" | "POST"): this { this.method = method; return this; }
  addHeader(key: string, value: string): this { this.headers[key] = value; return this; }
  build() { return { url: this.url, method: this.method, headers: this.headers }; }
}
```

```
const req = new RequestBuilder().setUrl("/api/users").setMethod("POST").addHeader("A
```

Returning `this` from each method (a "fluent interface") is what enables the chained call style above.

16.9 When to Reach for a Pattern (and When Not To)

Patterns solve specific recurring forces (variability in algorithm, variability in creation, decoupling layers). Applying a pattern where the underlying force doesn't exist adds indirection without benefit — a violation of **YAGNI** (§17.7). Senior engineers introduce a pattern when a second/third concrete variant of something already exists or is clearly imminent, not speculatively on day one.

17. SOLID, DRY, KISS, YAGNI — Principles Applied With Real TS Code

17.1 S — Single Responsibility Principle

A class/module should have one, and only one, reason to change.

```
// Incorrect – Violates SRP – this class has THREE reasons to change:
// business rules, persistence, and email formatting.
class UserService {
  createUser(name: string, email: string) {
    if (!email.includes("@")) throw new Error("Invalid email"); // validation
    const user = { id: Date.now(), name, email };
    fs.writeFileSync("users.json", JSON.stringify(user));           // persistence
    console.log(`<html>Welcome ${name}</html>`);                     // presentation/no
    return user;
  }
}

// Correct – Each concern is its own collaborator with one reason to change
class UserValidator { validate(email: string) { if (!email.includes("@")) throw new
class UserRepository { save(user: User) { /* persistence only */ } }
class WelcomeEmailSender { send(user: User) { /* email formatting/sending only */ }

class UserService {
  constructor(
    private validator: UserValidator,
    private repo: UserRepository,
    private mailer: WelcomeEmailSender,
```



```

) {}
createUser(name: string, email: string) {
  this.validator.validate(email);
  const user = { id: Date.now(), name, email };
  this.repo.save(user);
  this.mailer.send(user);
  return user;
}
}

```

17.2 O — Open/Closed Principle

Software entities should be open for extension, but closed for modification.

```

// Incorrect – Every new shape requires editing this function – closed for extension
function area(shape: { type: string; [key: string]: any }): number {
  if (shape.type === "circle") return Math.PI * shape.radius ** 2;
  if (shape.type === "square") return shape.side ** 2;
  throw new Error("Unsupported shape"); // grows forever as shapes are added
}

// Correct – New shapes are added by writing a new class – no existing code is touch
interface Shape { area(): number; }
class Circle implements Shape { constructor(private r: number) {} area() { return Ma
class Square implements Shape { constructor(private s: number) {} area() { return th
class Triangle implements Shape { constructor(private b: number, private h: number)

function totalArea(shapes: Shape[]): number {
  return shapes.reduce((sum, s) => sum + s.area(), 0);
}

```

17.3 L — Liskov Substitution Principle

Subtypes must be substitutable for their base types without breaking correctness.

```

// Incorrect – Classic LSP violation – Square "is-a" Rectangle geometrically, but no
class Rectangle {
  constructor(protected width: number, protected height: number) {}
  setWidth(w: number) { this.width = w; }
  setHeight(h: number) { this.height = h; }
  area() { return this.width * this.height; }
}

class Square extends Rectangle {
  override setWidth(w: number) { this.width = w; this.height = w; } // breaks caller
  override setHeight(h: number) { this.width = h; this.height = h; }
}

```

```
function resize(rect: Rectangle) {
  rect.setWidth(5);
  rect.setHeight(10);
  console.assert(rect.area() === 50); // FAILS for Square – silently wrong for a sub
}

// Correct – Model them as siblings behind a shared contract instead of forcing an i
interface Shape2 { area(): number; }
class Rectangle2 implements Shape2 { constructor(private w: number, private h: numbe
class Square2 implements Shape2 { constructor(private s: number) {} area() { return
```

17.4 I — Interface Segregation Principle

Clients should not be forced to depend on methods they don't use — prefer many small, focused interfaces over one large one.

```
// Incorrect – A fat interface forces irrelevant implementations
interface Worker { work(): void; eat(): void; sleep(): void; }
class Robot implements Worker {
  work() { /* ... */ }
  eat() { throw new Error("Robots don't eat"); } // forced, meaningless implementat
  sleep() { throw new Error("Robots don't sleep"); }
}

// Correct – Segregated interfaces – implement only what's relevant
interface Workable { work(): void; }
interface Eatable { eat(): void; }
interface Sleepable { sleep(): void; }

class Human implements Workable, Eatable, Sleepable { work() {} eat() {} sleep() {}
class Robot2 implements Workable { work() {} }
```

17.5 D — Dependency Inversion Principle

High-level modules should not depend on low-level modules; both should depend on abstractions.

```
// Incorrect – UserService (high-level policy) directly depends on a concrete low-le
class SQLiteDatabase { query(sql: string) { /* ... */ } }
class UserService {
  private db = new SQLiteDatabase(); // tightly coupled, impossible to swap or mock cle
}

// Correct – Both depend on an abstraction; the concrete implementation is INJECTED,
```

```
interface Database { query<T>(sql: string, params?: unknown[]): Promise<T[]>; }

class UserService {
  constructor(private readonly db: Database) {} // depends on the interface, not a c
}

class PostgresDatabase implements Database { async query<T>(sql: string) { return []
class InMemoryTestDatabase implements Database { async query<T>() { return [] as T[]

const service = new UserService(new PostgresDatabase()); // composition root wires t
```

This is the theoretical foundation behind **dependency injection frameworks** (NestJS `@Injectable`, InversifyJS, Angular DI) — DI containers exist purely to automate the "wire up concrete implementations to abstractions" step at the composition root.

17.6 DRY — Don't Repeat Yourself

Every piece of knowledge must have a single, unambiguous, authoritative representation.

```
// Incorrect — The "shape of a user" is duplicated across three call sites — a chang
function createUser(id: number, name: string, email: string) { return { id, name, err
function updateUser(user: { id: number; name: string; email: string }) { /* ... */ }
function logUser(user: { id: number; name: string; email: string }) { /* ... */ }

// Correct — A single named type is the one source of truth
interface User { id: number; name: string; email: string; }
function createUser2(user: Omit<User, "id">): User { return { id: Date.now(), ...use
function updateUser2(user: User) { /* ... */ }
function logUser2(user: User) { /* ... */ }
```

Caveat — DRY is about knowledge duplication, not code duplication. Two pieces of code that look similar but represent *different business rules that happen to coincide today* should often stay separate — prematurely merging them ("DRY-ing" too aggressively) creates a false coupling that causes pain the moment the rules diverge. This nuance is what separates senior engineering judgment from mechanical rule-following.

17.7 KISS & YAGNI

- **KISS (Keep It Simple, Stupid)** — prefer the simplest design that correctly solves the actual problem. A generic, configurable, plugin-based abstraction for a feature with exactly one use case is over-engineering.

- **YAGNI (You Aren't Gonna Need It)** — don't build extensibility, generic type parameters, or configuration options for hypothetical future requirements. Add abstraction when a second real use case appears, not speculatively.

```
// Incorrect — YAGNI violation — a fully generic, pluggable "rule engine" for a sing
interface Rule<T> { evaluate(input: T): boolean; }
class RuleEngine<T> {
  private rules: Rule<T>[] = [];
  addRule(rule: Rule<T>) { this.rules.push(rule); }
  evaluateAll(input: T) { return this.rules.every(r => r.evaluate(input)); }
}

// Correct — KISS — a single, simple, direct function until real variability actual
function isEligibleForDiscount(user: User): boolean {
  return user.age >= 65 || user.membershipYears >= 5;
}
```

17.8 Composition Over Inheritance (Recap With SOLID Lens)

Composition (injecting small collaborators, §11.11) is a natural consequence of applying SRP + DIP together: each collaborator has one responsibility, and the composing class depends on abstractions rather than inheriting concrete behavior — this is why "favor composition over inheritance" is often described as "SOLID applied at the object-relationship level," not a separate, unrelated principle.

18. Error Handling Strategies

18.1 Custom Error Hierarchies

Plain `Error` loses domain meaning. Production systems define a typed hierarchy so callers can branch on *kind* of failure, not string-matched messages.

```
abstract class AppError extends Error {
  abstract readonly statusCode: number;
  abstract readonly code: string;
  constructor(message: string, readonly cause?: unknown) {
    super(message);
    this.name = new.target.name;
    Object.setPrototypeOf(this, new.target.prototype); // fixes `instanceof` across
  }
}
```

```

class NotFoundError extends AppError {
  readonly statusCode = 404;
  readonly code = "NOT_FOUND";
}

class ValidationError extends AppError {
  readonly statusCode = 400;
  readonly code = "VALIDATION_ERROR";
  constructor(message: string, readonly fieldErrors: Record<string, string>) { super }
}

class UnauthorizedError extends AppError {
  readonly statusCode = 401;
  readonly code = "UNAUTHORIZED";
}

```

```

function handleError(err: unknown) {
  if (err instanceof ValidationError) {
    return { status: err.statusCode, body: { code: err.code, fields: err.fieldErrors } }
  }
  if (err instanceof AppError) {
    return { status: err.statusCode, body: { code: err.code, message: err.message } }
  }
  // Unknown/unexpected error – never leak internals to the client
  console.error("Unhandled error:", err);
  return { status: 500, body: { code: "INTERNAL_ERROR" } };
}

```

18.2 The `Result` / `Either` Pattern (Errors as Values)

Throwing exceptions is invisible in a function's type signature — callers can't tell from the type alone that a function might throw. For **expected, recoverable** failures (e.g., "user not found," "invalid input"), many production TS codebases prefer modeling failure explicitly as a return value, using the discriminated-union pattern from §9.3:

```

type Result<T, E = AppError> =
  | { ok: true; value: T }
  | { ok: false; error: E };

function parseAge(input: string): Result<number, ValidationError> {
  const n = Number(input);
  if (Number.isNaN(n) || n < 0) {
    return { ok: false, error: new ValidationError("Invalid age", { age: input }) };
  }
  return { ok: true, value: n };
}

```

```

}

const result = parseAge("abc");
if (!result.ok) {
  console.error(result.error.message); // compiler forces you to check `.ok` before
} else {
  console.log(result.value);
}

```

Guideline for choosing between `throw` and `Result`: reserve `throw`/exceptions for truly *exceptional*, unrecoverable, or programmer-error conditions (invariant violations, unreachable code, infrastructure failures). Use `Result` for *expected* business-level failure paths that the caller is meant to explicitly handle — it makes failure part of the type contract instead of a hidden control-flow path.

18.3 Never Swallow Errors Silently

```

// Incorrect – Silently swallowed – the failure vanishes, debugging becomes forensic
try { await sendEmail(user); } catch {}

// Correct – At minimum, log with context; usually also re-throw or return a Result
try {
  await sendEmail(user);
} catch (error) {
  logger.error("Failed to send welcome email", { userId: user.id, error });
  throw new AppError("Email delivery failed", { cause: error });
}

```

18.4 Runtime Validation at Trust Boundaries (Critical, Often-Missed Practice)

TypeScript types are erased at runtime and provide zero protection against malformed data arriving from outside your program — HTTP request bodies, `JSON.parse` results, environment variables, third-party API responses, message-queue payloads. Casting them with `as SomeType` is a *lie* the compiler will happily believe. Production-grade code validates untyped input with a **runtime schema library** (Zod, Valibot, io-ts) at every trust boundary, and derives the static type *from* the schema (single source of truth — DRY):

```

import { z } from "zod";

const UserSchema = z.object({
  id: z.number(),

```

```

    name: z.string().min(1),
    email: z.string().email(),
  });

type User = z.infer<typeof UserSchema>; // static type derived FROM the runtime sche

async function getUser(id: number): Promise<User> {
  const res = await fetch(`/api/users/${id}`);
  const json: unknown = await res.json();
  return UserSchema.parse(json); // throws a descriptive ZodError if the shape is wr
}

```

Environment variables deserve the same treatment — never sprinkle raw `process.env.SOME_VAR!` (non-null assertion) throughout a codebase; validate them once, at startup:

```

const EnvSchema = z.object({
  DATABASE_URL: z.string().url(),
  PORT: z.coerce.number().default(3000),
  NODE_ENV: z.enum(["development", "production", "test"]),
});
export const env = EnvSchema.parse(process.env); // crashes fast at boot if misconfi

```

18.5 Fail Fast, Fail Loud

Prefer guard clauses at the top of a function over deeply nested conditionals, and prefer crashing on startup for unrecoverable configuration errors over limping along with `undefined` values that surface as confusing failures three layers deeper.

19. Testing TypeScript

19.1 The Testing Pyramid, Applied to TS Projects

Most test value-per-dollar comes from a large base of fast unit tests plus a thinner layer of integration tests around real I/O boundaries, with only a handful of E2E tests protecting critical user journeys.

19.2 Unit Testing With Jest/Vitest

```

// discount.ts
export function applyDiscount(price: number, percent: number): number {
  if (percent < 0 || percent > 100) throw new RangeError("percent must be 0-100");
}

```

```

    return price * (1 - percent / 100);
  }

// discount.test.ts
import { describe, it, expect } from "vitest"; // or from "@jest/globals"
import { applyDiscount } from "../discount";

describe("applyDiscount", () => {
  it("applies a percentage discount correctly", () => {
    expect(applyDiscount(100, 20)).toBe(80);
  });

  it("throws for out-of-range percentages", () => {
    expect(() => applyDiscount(100, 150)).toThrow(RangeError);
  });
});

```

19.3 Testability Through Dependency Injection

Because §17.5 (DIP) means collaborators are injected via constructor rather than hard-coded, they can be trivially substituted with test doubles:

```

class FakeClock { now(): Date { return new Date("2026-01-01T00:00:00Z"); } }

class SubscriptionService {
  constructor(private clock: { now(): Date }) {}
  isExpired(expiresAt: Date): boolean { return this.clock.now() > expiresAt; }
}

// test
const service = new SubscriptionService(new FakeClock());
expect(service.isExpired(new Date("2025-01-01"))).toBe(true);

```

Classes that construct their own dependencies internally (`new SQLiteDatabase()` inside the class body) cannot be unit tested without hitting real infrastructure — this is the single most common reason production codebases retrofit dependency injection after the fact.

19.4 Mocking With Types Preserved

```

import { vi } from "vitest";

interface EmailClient { send(to: string, body: string): Promise<void>; }

```



```
const mockEmailClient: EmailClient = { send: vi.fn().mockResolvedValue(undefined) };  
// mockEmailClient still satisfies the EmailClient interface – full type safety in t
```

19.5 Type-Level Testing

Beyond runtime behavior, you can assert that your *types* behave correctly — critical for generic utility types and public library APIs:

```
import { expectTypeOf } from "vitest";  
  
expectTypeOf(applyDiscount(100, 10)).toBeNumber();  
expectTypeOf<Pick<User, "id">>().toEqualTypeOf<{ id: number }>();
```

Libraries like `tsd` or Vitest's built-in `expectTypeOf` catch regressions where a refactor accidentally widens/narrows a public type signature — a class of bug that ordinary runtime tests cannot detect at all.

19.6 Testing Discriminated Unions & Exhaustiveness

```
function describeStatus(s: Status): string {  
  switch (s.status) {  
    case "success": return "OK";  
    case "error": return "Failed";  
    default: {  
      const _exhaustive: never = s; // if a new Status variant is added without update  
      throw new Error(`Unhandled: ${_exhaustive}`); // this LINE fails to compile –  
    }  
  }  
}
```

19.7 Integration Testing Boundaries

```
// Use a real (but ephemeral/containerized) database for repository-layer tests,  
// rather than mocking the database driver – mocking your own persistence layer  
// only tests that your mocks behave as you told them to, not real SQL behavior.  
describe("PostgresUserRepository (integration)", () => {  
  it("persists and retrieves a user", async () => {  
    const repo = new PostgresUserRepository(testDbClient);  
    await repo.save({ id: "1", name: "Test User", email: "t@example.com" });  
    const found = await repo.findById("1");  
    expect(found?.name).toBe("Test User");  
  });  
});
```

19.8 Coverage & Test-Quality Guidance

- Coverage percentage is a *lagging* proxy metric, not a goal — 100% coverage with no meaningful assertions is worthless. Focus on covering business rules and edge cases (empty arrays, boundary numbers, error paths), not on the percentage number itself.
- Test behavior/contracts, not implementation details — a good unit test should survive an internal refactor that doesn't change observable behavior; a test that breaks on every refactor is over-coupled to implementation.
- Prefer real objects over mocks when the collaborator is cheap and deterministic (e.g., a pure `DiscountCalculator`); reserve mocks for genuinely expensive/non-deterministic boundaries (network, time, randomness, filesystem).

20. Tooling, Linting, Build Systems & CI/CD for Production TS

20.1 Linting & Formatting

```
npm install -D eslint @typescript-eslint/parser @typescript-eslint/eslint-plugin pre
```

Key rules production teams enable beyond the defaults:

Rule	Purpose
<code>@typescript-eslint/no-explicit-any</code>	Bans <code>any</code> — forces <code>unknown</code> + narrowing instead
<code>@typescript-eslint/no-floating-promises</code>	Catches un-awaited/un-caught Promises — a top source of silent async bugs
<code>@typescript-eslint/no-non-null-assertion</code>	Flags <code>!</code> usage for review
<code>@typescript-eslint/explicit-function-return-type</code>	Requires explicit return types on exported functions
<code>no-var</code>	Bans legacy <code>var</code>
<code>eqeqeq</code>	Requires <code>===</code> / <code>!==</code> over <code>==</code> / <code>!=</code>

Run **Prettier** for formatting (line length, quotes, spacing) and **ESLint** purely for correctness/style rules — don't let ESLint fight Prettier over formatting; `eslint-config-prettier` disables the overlapping rules.

20.2 Pre-Commit Enforcement

```
npm install -D husky lint-staged
npx husky init
```

```
// package.json
"lint-staged": {
  "*.ts,tsx": ["eslint --fix", "prettier --write"]
}
```

Catching issues at commit time (rather than in CI, minutes later) shortens the feedback loop dramatically.

20.3 Build Tooling Choices

Tool	Role	When to Use
<code>tsc</code>	Type-checking (always) + emit (optionally)	Libraries; type-check gate in CI for any project
<code>esbuild</code> / <code>swc</code>	Extremely fast transpilation (types stripped, NOT checked)	App bundling where <code>tsc --noEmit</code> runs separately as the type gate
<code>Vite</code>	Dev server + bundler for frontend apps (uses esbuild/swc under the hood)	Frontend SPAs
<code>tsup</code>	Zero-config bundler for TS libraries (wraps esbuild)	Publishing npm packages
<code>ts-node</code> / <code>tsx</code>	Run <code>.ts</code> files directly without a separate compile step	Scripts, local dev, quick iteration

Critical production pattern: bundlers that use esbuild/swc **transpile** TypeScript (strip types) but do **not type-check** it — a type error can ship to production silently if `tsc` isn't run as a separate step. Every production CI pipeline must run `tsc --noEmit` as an explicit gate, independent of the bundler.

20.4 CI Pipeline (Representative)

```
# .github/workflows/ci.yml
name: CI
on: [push, pull_request]
jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: "npm" }
      - run: npm ci # reproducible install from package-lock.json
      - run: npm run typecheck # tsc --noEmit
      - run: npm run lint # eslint .
      - run: npm run test -- --coverage
      - run: npm run build
```

Order matters and is deliberate: typecheck → lint → test → build. Fail fast on the cheapest checks first (typecheck/lint are seconds; full test suites and builds are much slower) so a broken PR gets feedback as quickly as possible.

20.5 Monorepo Considerations

For multi-package repos (npm/pnpm/Turborepo/Nx workspaces), use **TypeScript Project References** (`composite: true` per package + `references` pointing to dependency packages) so `tsc --build` incrementally recompiles only what changed, and each package gets its own `.d.ts` output consumable by sibling packages without a full rebuild.

20.6 Semantic Versioning & Public API Discipline

For published libraries, `.d.ts` output **is** your public contract. Treat any change to an exported type as a potential breaking change subject to semver rules — adding a new required property to an exported interface is a **major** version bump even though it might "just be a type change," because it can break every consumer's compilation.

21. Performance, Compiler Internals & Scaling Large Codebases

21.1 Compile-Time Performance

- `skipLibCheck: true` — skips re-checking every dependency's `.d.ts` files; a large compile-time win with negligible safety cost in practice (dependencies are assumed to type-check themselves before publishing).
- **Project References** + `incremental: true` — enables `.tsbuildinfo` caching so unchanged files aren't rechecked on every build.
- **Avoid deeply recursive/huge union/conditional types** in hot paths of your type definitions — the compiler has real algorithmic limits (recursion depth caps, instantiation limits) and pathological generic code can measurably slow down `tsc` and editor responsiveness across a whole team.
- **Prefer `interface` over large `type` intersections** for object shapes that are checked often — interfaces are cached/named internally in a way that tends to check faster than resolving deep intersection chains repeatedly.

21.2 Runtime Performance (What TS Does and Doesn't Affect)

Types are erased — **TypeScript itself adds zero runtime overhead**. Runtime performance is purely a function of the emitted JavaScript and your algorithms/data structures, same as any JS codebase:

- Choose the right `target` (§2.3) — targeting too old an ES version forces the compiler to down-level modern syntax (e.g., generators, classes) into more verbose, slower polyfilled code; target the newest runtime you actually support (modern Node LTS, or your supported browser matrix).
- Enums (§8.1) that are not `const enum` generate an extra runtime object — negligible for most apps, but worth knowing at scale (thousands of enum members across a bundle).
- Excess use of getters/setters or Proxy-based reactivity (common in some state-management libraries) has real runtime cost per access — profile before assuming it's free.

21.3 Scaling Type-Checking Across Large Codebases

- Split large monorepos into TypeScript **project references** so IDE responsiveness and `tsc --build` scale with *changed* code, not total code.
- Avoid barrel files (§13.3) that re-export hundreds of modules from one entry point in large packages — they force the compiler (and bundlers) to load far more than is needed for a single import.
- Periodically audit for `any` leakage using `@typescript-eslint/no-explicit-any` in CI — `any` is contagious (anything derived from an `any` becomes `any`), and left unchecked it silently erodes type coverage across a growing codebase.

21.4 Architectural Guidance for Large Systems

- **Composition root pattern** — construct the full dependency graph (concrete implementations wired to abstractions, §17.5) in exactly one place (e.g., `main.ts` / `bootstrap.ts`), keeping every other module ignorant of concrete implementations.
 - **Bounded contexts / feature modules** (§13.7) — keep domain boundaries explicit; cross-feature imports should go through a clearly exported public surface, not reach into another feature's internals.
 - **Anti-corruption layer** — when integrating an external system (legacy API, third-party SDK) whose data model doesn't match your domain, wrap it behind an Adapter (§16.7) so external model changes don't ripple through your core domain types.
 - **Immutable-by-default data flow** — prefer `readonly`, `ReadonlyArray<T>`, and returning new objects over mutation; it drastically reduces a whole class of "who mutated this and when" bugs in large teams, at the cost of being slightly more verbose.
-

22. Interview Cheat Sheet (Expanded)

22.1 Frequently Asked Interview Questions

1. What is TypeScript and how does it differ from JavaScript?
2. What are the benefits of using TypeScript in a production codebase?
3. Explain `any` vs `unknown` vs `never` vs `void`.
4. What's the difference between `interface` and `type`? When would you choose one over the other?
5. What are Enums, and why might you prefer a `const`-object + union-literal pattern instead?
6. What is a type assertion (`as`), and how does it differ from a type guard?
7. What are generics? Give a real example beyond `identity<T>`.
8. What is the difference between `keyof` and `typeof` as type operators?
9. How does `async` / `await` relate to Promises and the event loop?
10. What is a discriminated union, and how does exhaustiveness checking work?
11. Explain structural typing vs nominal typing, and how "branded types" simulate nominal typing in TS.
12. What is the Dependency Inversion Principle, and how does it show up in TypeScript code?

13. Why is `strictNullChecks` important, and what class of bugs does it eliminate?
14. What happens to TypeScript types at runtime?
15. How would you validate an untrusted API response in a type-safe way?

22.2 Common Mistakes (What Senior Reviewers Flag in PRs)

- Using `any` as a default escape hatch instead of `unknown` + narrowing.
- Not enabling `strict` mode (or worse, disabling individual strict flags to "make errors go away").
- Ignoring `null`/`undefined` handling instead of modeling absence explicitly.
- Overusing `as` type assertions instead of writing/using a proper type guard.
- Mutating data in place instead of using `readonly`/immutable patterns.
- Reaching for classes/interfaces for every object instead of plain types where appropriate — over-engineering simple data shapes.
- Overusing the non-null assertion `!` to silence the compiler instead of handling the null case.
- Forgetting to handle errors in `async` code (missing `try/catch`, or an un-awaited Promise that produces an `unhandledRejection`).
- Deep inheritance chains where composition + interfaces would be more flexible and testable.
- Casting untrusted external JSON directly to a type without runtime validation (§18.4).

22.3 Best Practices Checklist

- Enable `strict: true` (and ideally `noUncheckedIndexedAccess`, `exactOptionalPropertyTypes`) in `tsconfig.json`.
- Prefer `interface` for object/class shapes; `type` for unions, tuples, and mapped/conditional types.
- Use `unknown` (never `any`) at true type boundaries, narrowed via guards or a schema validator.
- Prefer `readonly` for data that shouldn't be mutated by the receiver.
- Let TypeScript infer local variable types; explicitly annotate exported function signatures.
- Handle errors explicitly in `async` code; never swallow exceptions silently.
- Keep functions and classes small and single-purpose (SRP).
- Depend on abstractions (interfaces), not concrete implementations (DIP) — inject collaborators.
- Validate all external/untrusted data at runtime, not just at the type level.

22.4 Naming Conventions

Element	Convention	Example
Variables	camelCase	<code>userName</code>
Functions	camelCase	<code>getUserData()</code>
Classes / Interfaces	PascalCase	<code>UserService</code> , <code>UserRepository</code>
Types / Enums	PascalCase	<code>UserType</code> , <code>StatusEnum</code>
Constants (true, global constants)	UPPER_SNAKE_CASE	<code>MAX_LIMIT</code>
Generic type parameters	Single uppercase letter, or <code>T</code> -prefixed for clarity in complex code	<code>T</code> , <code>K</code> , <code>TEntity</code>
Files	kebab-case	<code>user-service.ts</code>
Folders	kebab-case	<code>user-service/</code>

Consistent naming — enforced by a shared ESLint config, not tribal knowledge — is what keeps a codebase readable as team size grows.

22.5 Recommended Production Folder Structure (Recap)

```
src/
├─ assets/           # images, icons, fonts
├─ modules/          # feature-based folders: controller/service/repository/types pe
├─ shared/
│   ├─ types/        # cross-cutting shared types
│   ├─ utils/         # pure helper functions
│   └─ errors/        # AppError hierarchy
├─ config/           # validated environment/config (Zod-parsed at boot)
└─ index.ts          # composition root / entry point
```

Why this structure: separates concerns clearly, scales cleanly as features are added/removed, improves reusability of shared code, and makes team collaboration easier since features are self-contained.

23. One-Page Quick Revision Sheet

23.1 Important Syntax

```
let name: string = "Mahesh";
const PI: number = 3.14;
function add(a: number, b: number): number { return a + b; }
const add2 = (a: number, b: number): number => a + b;
interface User { id: number; name: string; }
type ID = number | string;
class User2 { constructor(public name: string) {} }
enum Status { Active, Inactive }
const nums: number[] = [1, 2, 3];
const user: [number, string] = [1, "Mahesh"];
let data: any;
let value: unknown;
let len = (value as string).length;
```

23.2 Most Used Keywords

let · const · function · return · if / else · for · while · switch · break · continue ·
class · interface · type · enum · extends · implements · public / private / protected /
readonly · static · async · await · try / catch / finally · throw · import / export /
from · keyof · typeof · infer · satisfies · as

23.3 Learning Roadmap Recap

1. Basics → 2. OOP Concepts → 3. Advanced Types → 4. Modules → 5. Async JS → 6. Patt

Stage	Covers
1. Basics	Variables, primitive types, functions, arrays, tuples
2. OOP Concepts	Classes, interfaces, inheritance, polymorphism, encapsulation
3. Advanced Types	Generics, utility types, conditional/mapped types
4. Modules	Import/export, project structure, path aliases
5. Async JS	Promises, <code>async</code> / <code>await</code> , error handling, combinators
6. Patterns/SOLID	GoF patterns, SOLID, DRY/KISS/YAGNI
7. Testing/Tooling	Unit/integration tests, ESLint, CI pipelines

Stage	Covers
8. Build & Ship	Compiler config, bundlers, performance, monorepo scaling

23.4 One-Page Summary

- TypeScript = JavaScript + a static structural type system, fully erased at compile time.
- It catches errors early, documents intent, and enables safe large-scale refactoring.
- `unknown` + narrowing is the type-safe replacement for `any`.
- Discriminated unions + exhaustiveness checks are the idiomatic way to model variant state.
- SOLID principles map directly onto everyday TS decisions: small classes (S), extend-don't-edit (O), honest subtyping (L), focused interfaces (I), inject abstractions (D).
- Types provide **zero runtime protection** — validate all external data with a schema library at trust boundaries.
- Prefer composition, dependency injection, and immutability by default for testable, maintainable, large-scale systems.
- Type-check (`tsc --noEmit`) as an explicit, separate CI step — bundlers like esbuild/swc do not type-check.

23.5 Final Notes

Consistency + practice + real projects = mastery. Build small projects deliberately using `strict` mode, layered architecture, dependency injection, and tests from day one — the habits formed on a small project are exactly the habits that scale (or fail to scale) on a large one.

Keywords: TypeScript tutorial, learn TypeScript, TypeScript for beginners, advanced TypeScript, TypeScript best practices, TypeScript generics, TypeScript interfaces vs types, TypeScript design patterns, SOLID principles TypeScript, TypeScript async await, TypeScript interview questions, TypeScript project structure.



Top comments (0)

[Code of Conduct](#) • [Report abuse](#)